FRED TALKS

16th July 2026

CONTENTS

Agentic search - retrieval, harness, or model?

DOUG TURNBULL · 09 JUN 2026 · SOURCE

Agentic search gets interesting when agents do not know how to find the right answer.

Oh, the agent might think it knows. It might confidently BS us. But the agent's poor domain intuition steers itself astray.

Agents make false assumptions about what *our* users think is relevant. Our fashionista users think "red shoes" should return high-heels. When I worked at one company ABE wasn't a president, it was an A/B testing tool. Agents need context to know these things - and context engineering needs agentic search

Confusingly, depending who you talk to, agentic search can mean one of three distinct implementation patterns:

- **Retrieval-centric** - just build good search so agents can use it to fill in missing context
- **Harness-centric** - steer the agent towards needed context, even with bad search
- **Model-centric** - fine tune an LLM to know how to search *our* data

In this post, I'll walk through each. With a bit more breadth, you might appreciate which flavor your colleagues seem to mean when they say "agentic search"

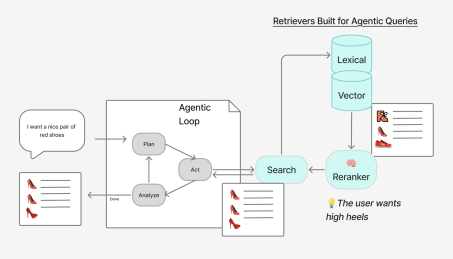
Retrieval-Centric Implementation

When frontier models don't know, they search. Ask ChatGPT a question about news, it'll search. Ask it about a very specific technical problem, it'll search. During training, LLMs see *search* examples as a technique to learn what it doesn't know. To find what it needs.

Therefore, we just need to build good search to solve any conceivable query from an agent.

Let's assume we run an e-commerce catalog. For us, when users search "red shoes" they mean "red high heels". But the agent doesn't know that. Luckily it asks *search*. Below we see it returns the right results.

Below we see initial retrieval to lexical / vector backends pulls back some reasonable, if naive, “red shoe” candidates. Still that’s not quite right. Luckily the reranker shapes the results towards our understanding of that intent.



Of course we might have other components here - query understanding, diversity, custom embedding models, and more.

The important point is that *search leads the agent by the nose* towards what’s relevant. We assume search can define what a good “red shoe” is, overriding the agent’s perspective.

Most teams build retrieval-centric approaches with classic RAG. Chunks of answers, and embeddings trained to recognize them as answers.

Unfortunately, answers don’t always look tied to the question. For example:

Question:

> Synopsis of the book Ubik

Answer:

> By the year 1992, humanity has colonized the Moon and psychic powers are common. The protagonist, Joe Chip, is a debt-ridden technician working for Runciter Associates, a “prudence organization” employing “inertials”—people with the ability to negate the powers of telepaths and “precogs”—to enforce the privacy of clients.

If you don’t know the book Ubik, it’s not clear that this answers the question. The agent says “cool story bro” and ignores the info.

Search like this actually is divorced from the web search trusted by fronteir models. The web contains titles, headings, and other elements placing the answer in context.

The big downside? Search remains hard. We're not Google.

Most importantly, we don't have perfect search like Google or Bing. Even with good search - almost nobody builds *google quality results.* And since agents trust search - simple distractors in retrieval can, as Lester Solbakken says, easily confuse reasoning.

Harness-centric implementations

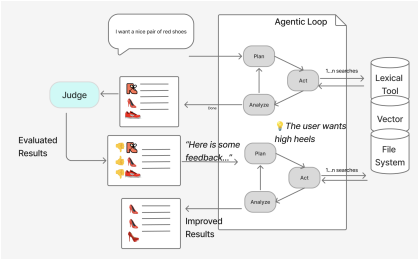
Who should be in charge? Should the agent manage the search process? Or should search drive the agent?

Moving the emphasis to the harness, we put the agent in charge.

Imagine stripping search tools down to core retrieval primitives. Just a BM25 backend. Or a filesystem with CLI tools. We tell the agent find to what it needs with these untuned tools. The agent might struggle more, but hey, its smart, it can get figure it out. Right?

Still, the agent might find what it thinks is relevant, as it does in the image below. Sadly that's not what's actually relevant. To help the agent, we inject external knowledge. We let a judge direct the agent, correcting its mistakes, and guiding it towards better search strategies.

So when the agent returns results, it's not the "user" that receives the candidates. Instead a judge labels results as relevant or not. The agent gets the hint, finding results similar to those labeled relevant. Avoiding those labeled irrelevant. To steel from Jo Kristian Bergem, this is relevance feedback on steroids.



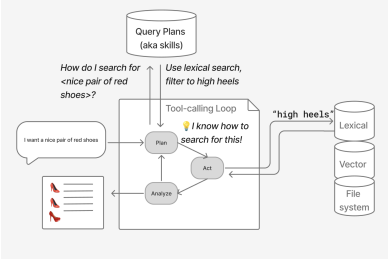
This works. Look at the ESCI dataset. If I have an oracle labeling with judgments from these datasets, I get quite significant improvements.

Variant	NDCG@10	Description
ESCI BM25	0.2895	Simple BM25 weighing name / description
ESCI agentic	0.4101	GPT5-mini tool calling loop w/ BM25+e5 embeddings tools

ESCI agentic w/ oracle	0.5843	GPT5-mini tool calling loop w/ BM25+e5 embeddings tools. Judge responds with feedback once

Here the second row we trust the agent's interpretation of queries. That provides a large gain over BM25. But inject more domain knowledge via a judge (the third row) and agent performance improves dramatically.

We can guide agents with information ahead of time too. For example, I've seen teams take inspiration from skills in coding agents. Imagine a targeted query plan that tells the agent how to use it's tools to solve a specific problem.



Optimizing content helps harnesses

We help agents even further by optimizing content for findability. That's the trick we've played on ourselves with coding agents. We write docs to explain to agents why something's useful. We don't just rely on that dumb chunk, we document the purpose of the knowledge:

> /books/scifi/ubik.txt

> ## About Book Ubik Ubik is a book by Philip K. Dick
>
> ## Synopsis
>
> By the year 1992, humanity has colonized the Moon and psychic powers are common. The protagonist, Joe Chip, is a debt-ridden technician working for Runciter Associates, a "prudence organization" employing "inertials"—people with the ability to negate the powers of telepaths and "precogs"—to enforce the privacy of clients.

Unlike a naive chunk, this bit of information has purpose. It's clear, when its in the context, what problem it solves.

By organizing + cataloging information we've done what every search developer wishes their content team would do: actually optimize content to be findable.

The downside: cost

The major downside here? Token costs. The exploration here requires more than just issuing a search and retrieving one set of results. It's an agent's exploration of the environment. And that exploration starts fresh with every new context window.

Model-centric approach

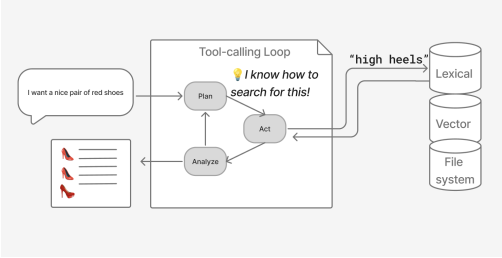
What if we literally fine-tuned the LLM to search efficiently?

The harness based approach takes a long, winding path towards relevant results. With every fresh context, it wastes calls relearning what works over-and-over. It's costly and slow.

Could we train a model that doesn't repeat the same mistakes with our tools?

Our judge helps label some of the agent's reasoning / tool-calling paths as useful. And label others less so. Once you have labeled traces, you can fine-tune an open weights model to proactively, efficiently make the right tool calls given the input.

Below, for example, the model knows exactly how to search for our "red shoes" query given its extensive fine tuning.



Now we've built a model targeting the search task. We don't tune our retrievers much. Instead we continue to use the simpler / dumb search tools.

Even cooler, the agentic search model's behavior can change just by adjusting the system prompt. "This is an e-commerce search task, optimize for top 10 ranking" might be different from "You're retrieving chunks for financial research, maximize diversity and relevance at the same time."

Further the models can stay smaller. By just focusing no the search part - not the entire domain problem - they can shrink to a manageable size allowing them to be self hosted.

The downside? Early days

The OG in the space is SID. Other contenders like Glean's Waldo model have sense entered. They're all promising, but not widely adopted. Many unknown unknowns exist. In a sense, we don't yet know the upside or downsides yet. I suspect though this paradigm has tremendous promise in the years ahead.

In other words - stay tuned!

The alchemy of ambiguity

People use "agentic search" confusingly and inconsistenly. They focus on their part of the problem. That means we have a collection of solutions like enriched base metals - and its unclear what alchemy brings them together.

We're converging on a fascinating mix of easier to organize data (like PageIndex). I'm seeing patterns in harness design that mirror coding agents - hooks / evaluators that steer agents. Or "skills" that guide them a priori. Much like with coding - the agents eat the harnesses. When a successful pattern emerges, agents, in our case agentic search models, train to memorize these patterns.

Retrieval also seems to be moving towards agent-centric approaches. Agents have their own specific, unique retrieval patterns. We're finally seeing late interaction + learned sparse retrieval approaches have their day from companies like LightOn and MixedBread

We're swimming in an interesting retrieval primordial goop, what comes out may be an obvious combination - or look nothing like the component parts. Exciting times!

Enjoy softwaredoug in training course form!

Starting May 18!

Signup here - http://maven.com/softwaredoug/cheat-at-search

I hope you join me at Cheat at Search with Agents to learn use agents in search. build better RAG and use LLMs in query understanding.

How not to forget what matters

HENRIK KARLSSON · 07 JUL 2026 · SOURCE

Read distraction-free on Substack

1.

Every few months I will read a tweet, or have a conversation, that makes me feel *this is important, I must remember this.* Often, these epiphanies are accompanied by a sense that *I actually know this already*, it had just somehow slipped my mind.

And for a few days, I do remember: my life shimmers with a new intensity, and I live the truth of what I grasped. But then, inevitably, the conveyor belt of things to pay attention to keeps churning, and my mind gets filled with small problems I need to solve, or new epiphanies or random noise, like news, and the shining fades from my eyes—I regress to being the same person as ever.

The Latin word for the tendency to lose track of what matters in the cacophony of things that attract our attention is *stultitia*. "Stultitia," writes Michel Foucault in "Self-writing,"

> is defined by mental agitation, distraction, change of opinions and wishes, and consequently weakness in the face of all the events that may occur; it is also characterized by the fact that it turns the mind toward the future, makes it interested in novel ideas, and prevents it from providing a fixed point for itself in the possession of acquired truth.

You can't just read a blog post about high agency, get filled with a sense of possibility, and become, from then on, an agentic person. As John Gray puts it in his monograph on J.S. Mill, our character is "a cluster of habitual willings." For changes to our behavior to become permanent, we must become different people.

In the same way that it is not enough to make a *resolution* that you will learn the piano, it is not enough to *realize* that when the kids act out, you shouldn't lose your temper but slow down, listen, and regulate their nervous systems with the help of yours. Imagine how good a person I would be if having insights were enough! But reacting to the frustrations of your children with calm and curiosity is a skill as much as playing the piano is—and as with the piano, the act of learning it requires rewiring your nervous system through sustained attention and practice. Realizing the value of acting in a certain way might give you a temporary motivation to do it. But in order to actually live in accordance with what you believe in long-term, you must make it a habit.

And this is much harder than making a habit out of playing the piano. When you're trying to make something like piano practice a habit, the standard advice is to chain it onto some already existing habit—to practice immediately after you brush your teeth in the morning, for example, or after you change out of your work clothes in the afternoon. Chaining the new habit to an already existing one provides a predictable trigger that helps remind you to practice. But the habits that make up our characters often do not follow a predictable schedule like this. I never know, for instance, when our children will act out (except that it will usually be when I'm least capable of handling it with grace—whenever both they and I are unusually hungry and tired). The conflicts seem to come out of nowhere, so I have to, somehow, *always* be ready to act in the proper way. I need to have the right reaction "ready at hand" (*procheiron*), as the Greek-

philosopher-Roman-slave Epictetus put it. If Johanna and I talked about how we want to deal with the kids' conflicts the night before, I will nearly always handle the situation well. The problem is to keep it top of mind.

2.

During the first two centuries of the Roman Empire, there spread a practice known as *hypomnēmata*, a type of notetaking system, used as a tool for meditation, [1] in which the writer would store quotes from books they had read. Each day, often in the morning, the notetaker would open their notebook and look for a passage relevant to something they were struggling with, and then they would meditate on that—unpacking it, making the idea top of mind, ensuring it was alive in them. If they needed courage, for instance, they could meditate on an anecdote that made it real for them what it meant to act bravely. The idea was that over time, the insights they gathered by reading would be transformed into character, something deeply ingrained in their way of thinking and seeing and acting.

This was, as I understand it, an exercise designed to combat the problem I outlined above. Meditating on what matters is a simple habit, which you can chain onto your morning routine, but it reinforces the habits you can't plan, the habits that make up your character. It was, in the words of the French classicist Pierre Hadot, a spiritual exercise—an *exercise* because it required work and discipline, *spiritual* because it engaged the whole person, not just their intellect, but their emotions and their moral character. It was an attempt to treat the formation of character as a skilled practice, as something you can deliberately train and improve through targeted exercises.

The most famous example of this practice is the meditations Marcus Aurelius wrote in his tent as plague swept through the camps during the military campaigns along the Danube River, but it seems to have been a fairly widespread practice among the "cultivated class." A suggestion repeated in several popular manuals for living was that you should collect every snippet of thought that deeply inspires you to live in a more ethically true way and then, in the pre-dawn hour, look through your hypomnēmata to find passages relevant to your current situation—insightful quotes, examples, actions you had witnessed, notes from conversations you'd had, and so on [2] —and meditate, in writing, on those that help you orient toward your current challenges, until you feel inspired to act in the proper way. [3]

What could be an example? Today, June 14, I woke up with a headache after having slept with my neck at an odd angle and so didn't feel like working. Having procrastinated for an hour or so, while Johanna tried to get me to start the day, I recalled principle 23 from a list Nabeel S. Qureshi has compiled for himself:

> Doing things is energizing, wasting time is depressing. You don't need that much 'rest'.

Walking around the farm with a cup of coffee, [4] reminding myself of the truth of this observation, I gained a sliver of motivation, enough to throw myself into rewriting this essay—and now my headache has lifted, and I feel excited. For more examples of things one might meditate on, I suggest looking at Nabeel's list. I also like to meditate on stories that make real to me some ethos I aspire to, such as the story of how Werner Herzog, dead broke while scouting for locations for *Nosferatu* in Brittany, happened upon a field of menhirs and decided to abandon his work to stay at the field for as long as necessary to solve the mystery of how the giant stones had been erected—a story that makes it visceral to me what it means to "take your curiosity seriously."

The hypomnēma was a mechanism for centering your mind on what matters, and for gradually refining your understanding of what matters. It was, to use a word from Plutarch, a tool for *ethopoiesis* (*ethos* meaning "character" and *poiesis* "making"), a tool for turning truth into character. By meditating daily on sentences that made it real to you how you wanted to live, you would remember to do the right thing—you would remember to practice it during the day (simply thinking about it is not going to change you). And gradually, you would become that sort of person. You wouldn't even need to remind yourself. Your principles would have become your character. You would have developed expertise in the skill of holding yourself in a better way.

3.

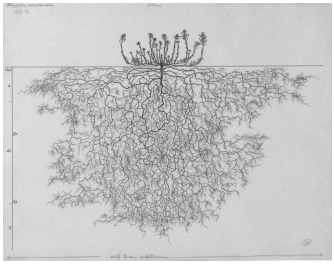
I first came across the idea of using hypomnēmata in this way eight years ago when a friend sent me a copy of Foucault's "Self-writing." In the essay, Foucault talks about hypomnēmata and other modes of reading and writing used to fashion a self during the Hellenistic and Imperial eras. I didn't pick up the practice in its full form. But rereading the essay now, I realize how much it has influenced how I write: my essays are (often) meditations I do in order to deepen core ideas I want to live by, and to strengthen parts of myself I want strengthened. [5] (Writing treatises and letters was a common way of internalizing the content of the hypomnēmata.) And this practice has been transformative for me. I have often noticed that my experience of reality improves if I write and think about something.

But it strikes me now that the practice Foucault wrote about was probably more transformative than what I've ended up doing. Essay writing is incredibly time-consuming, and a lot of that time is spent on things that aren't self-transforming: I spend less time reshaping my mind than I spend solving literary-technical problems that help me write more functional and beautiful

essays, for the joy of the craft and for the benefit of readers. Another limitation of my practice is that when an essay is done, I move on. The ideas—though they have been much deepened and more firmly lodged in my mind—fall out of attention and start to fade.

There is an element of self-deception involved here. I like to write essays, so it is comforting to think of it as a powerful practice, something that helps me live more fully and grow as a person. But if I look at it soberly, it is clear to me that essay writing is not a practice that is ideal for the purpose of ethopoiesis. [6] It is common to think that what we do achieves what we want it to achieve, even if there is no evidence for it. There are many practices that promise to transform and improve us—therapy, meditation, psychedelics—, but that branding doesn't mean that they actually do much for us: it is common to see people use these techniques for years without any obvious progress on their problems. If you want to achieve a particular outcome, it is important to start from that goal and evaluate which practices actually help you.

The most important ideas we need to return to weekly, even daily. Essay-writing, then, is not a functional substitute for having a practice that keeps the important truths top of mind, day after day. But it did help me reach that conclusion.



How to think in writing

Credits

This essay is based on a fragment from a longer essay on spiritual exercises that I wrote. That essay didn't work, and Johanna suggested using a few of the paragraphs for a new essay. She also suggested the main argument and general outline of the body of this essay, reorganized the second draft to shift the emphasis to what we came to see as the main idea, and corrected several faulty lines of argument I made.

The ideas were influenced by reading Pierre Hadot's Philosophy as a Way of Life *and Michel Foucault's* The Hermeneutics of the Subject, The Care of the Self, *and "Self-Writing." I was influenced by several conversations with Michael Nielsen and his notes on Tanya Luhrmann's* How God Becomes Real, *as well as a conversation with Alexander Obenauer about his practice. Claude Fable 5 (RIP) corrected four factual errors.*

1

The practice of keeping hypomnēmata had already existed for more than 400 years at this point, but the practices around it shifted significantly around this era, as I understand it, from a more conventional type of notetaking into the more specialized type of meditation described in this essay.

2

How do you actually keep these notes? A lot of people have insights and write them into Apple Notes, but then they never look at them again, and it becomes a mess after a while, so when they need the idea, it is nowhere to be found. I'm not sure exactly how the Romans managed to find their way back to the right wax tablet or papyrus. I guess the easiest thing is to just keep a single document for the most important ideas and stories, with short memorable concept handles (länk) that help you remember each. If you can't be bothered making your notes structured, though, LLMs are powerful enough now that they can trawl through your messy notes and find the relevant quotes. Just give Claude Code or Codex access to your notes and ask it to find segments relevant to whatever you are thinking about. This will not be as powerful as doing the work yourself, but it is better than just forgetting.

3

I read somewhere that when meditating on their hypomnēmata, the Romans and Hellenistic Greek would also set intentions for themselves, principles they want to live by that day. I can't find the source for this claim now, but it resonated with something Johanna often tells me: it is much easier to hold the line if you've articulated your position to yourself beforehand. If, for example, I'm on a call with someone who wants to propose a project, I will often get caught up in their excitement, or feel uncomfortable setting the right boundaries—I often don't even know what the right boundaries are, because there is too much to consider for me to figure out my position in real time—and so I will say yes to things I shouldn't. But if I have thought things through beforehand, and have articulated to myself what I'm trying to achieve, and what my principles are, I can more easily see if this is in line with what I want, and I can clearly articulate my position. I wish I did this more often. Or at least remembered the principle to always ask for time to think things through on my own before committing.

4

Seneca and Epictetus would have insisted on making meditating in writing. The argument is that writing engages you more deeply, transforming what you have read “into tissue and blood” (in vivres et in sanguinem). Using a an oft-repeated metaphor, Seneca writes, “We should imitate bees, who wander and pluck from flowers suited for making honey, then arrange and distribute through the combs what they have brought in... We must digest what we have read; otherwise it passes into the memory, not into the mind.” Food, he says, is a burden so long as it stays what it was; only when it changes into blood and tissue does it become strength. It might feel like I understand a topic without writing about it, but if I earnestly try to unpack my thinking in writing, my ignorance is revealed, and my thinking transformed.

5

I’m not sure how much of the shift in my writing came directly from reading Foucault: it has been something that I’ve iterated myself toward. I’ve gradually written more with the intention of shaping my character and my attention, and this has been driven by the experience of deepened presence this has afforded me. But I’m pretty sure the ideas from that essay provided a template that helped me make sense and direct the evolution of my writing. I know I have often returned to the essay, and have used Plutarch’s term ethopoetical writing to make sense to what I’m trying to do.

6

What it can do well, though, is to push deep into an idea and to restructure your thoughts about something. It is like an intervention where you want to shake things up properly, a three week session where you think and think and think about the same thing all the time. It is, I think, a useful complement to a daily practice.

A guest post by

Johanna Karlsson

x

Subscribe to Johanna

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · SOURCE

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the M×N integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it’s thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We’re already seeing this in adoption. The MCP SDKs recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including Claude Cowork, Claude Managed Agents, and channels in Claude Code.

As MCP continues to support production agentic systems, we’re sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our directory, used by millions of people every day. From working closely with enterprises and developers building on the protocol, we’ve spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single create_issue_from_thread tool beats get_thread + parse_messages + create_issue + link_attachment. See writing effective tools for agents to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. Cloudflare's MCP server is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

MCP Apps is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

Elicitation lets your server pause mid-tool call to ask the user for input. Form mode sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. URL mode hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest MCP spec supports CIMD (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

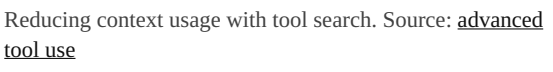
Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. Vaults in Claude Managed Agents covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents (*clients*) connect to and work with tools and data sources they need (*servers*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

Tool search defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our testing, tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.



Reducing context usage with tool search. Source: advanced tool use

Process tool results in code with programmatic tool calling

Programmatic tool calling processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly 37% on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See *advanced tool use* for the full breakdown.

Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our data plugin for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: Extending Claude's capabilities with skills and MCP servers

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. Canva, Notion, Sentry, and more do this today in Claude, listing the skill next to their connector in our web directory.

To make that pairing portable across every client, the MCP community is actively working on an extension for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.